

## 6.10 Computation

The formulas  $\mathbf{M} = \mathbf{X}(\mathbf{X}\mathbf{T}\mathbf{X})^{-1}\mathbf{X}\mathbf{T}$  and  $\hat{\boldsymbol{\beta}} = (\mathbf{X}\mathbf{T}\mathbf{X})^{-1}\mathbf{X}\mathbf{T}\mathbf{y}$  are the right way to *reason* about least squares and the wrong way to *compute* it. There are two problems.  $\mathbf{M}$  is  $n \times n$ , so forming it costs  $O(n^2)$  memory for a quantity that is never needed in full. And forming  $\mathbf{X}\mathbf{T}\mathbf{X}$  squares the condition number of the problem, which can throw away half of the available floating-point digits before any equation is solved. This section shows how the geometry itself suggests the fix, which is to build an orthonormal basis for  $\mathbb{C}(\mathbf{X})$ . Chapter 10 develops the numerical linear algebra in full.

### 6.10.1 QR: Gram–Schmidt as a factorization

Apply Gram–Schmidt (Proposition 6.2.2) to the columns  $\mathbf{x}_1, \dots, \mathbf{x}_p$  of a full-column-rank  $\mathbf{X}$ . Each  $\mathbf{x}_j$  is a combination of  $\mathbf{u}_1, \dots, \mathbf{u}_j$  with coefficients  $r_{ij} = \mathbf{u}_i^T \mathbf{x}_j$  for  $i < j$  and  $r_{jj} = \|\mathbf{w}_j\| > 0$ . In matrix form,

$$\mathbf{X} = \mathbf{Q}\mathbf{R},$$

where  $\mathbf{Q} = [\mathbf{u}_1, \dots, \mathbf{u}_p]$  is  $n \times p$  with  $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}_p$ , and  $\mathbf{R}$  is  $p \times p$  upper triangular with positive diagonal. This is the **thin QR factorization**. Since  $\mathbb{C}(\mathbf{Q}) = \mathbb{C}(\mathbf{X})$ , Proposition 6.3.2 gives  $\mathbf{M} = \mathbf{Q}\mathbf{Q}^T$  at once, and the rest of least squares follows:

$$\hat{\mathbf{Y}} = \mathbf{Q}(\mathbf{Q}^T\mathbf{y}), \quad \mathbf{R}\hat{\boldsymbol{\beta}} = \mathbf{Q}^T\mathbf{y}, \quad \text{SSE} = \|\mathbf{y}\|^2 - \|\mathbf{Q}^T\mathbf{y}\|^2, \quad h_{ii} = \sum_k q_{ik}^2. \quad (6.10.1)$$

The coefficient equation holds because the normal equations become  $\mathbf{R}^T\mathbf{R}\mathbf{b} = \mathbf{R}^T\mathbf{Q}^T\mathbf{y}$ , and  $\mathbf{R}^T\mathbf{R}$  is nonsingular. It is solved by back substitution, with no matrix inverse. Nothing of size  $n \times n$  is ever formed.

In floating point, Gram–Schmidt as written in Proposition 6.2.2 gradually loses orthogonality among the computed  $\mathbf{u}_j$ . Production software computes the same factorization with Householder reflections, which are backward stable (Golub and Van Loan 2013; Higham 2002). The geometry does not change:  $\mathbf{Q}$  is an orthonormal basis for the model space.

### 6.10.2 Conditioning

The sensitivity of least squares to rounding is governed by the **condition number**

$$\kappa(\mathbf{X}) = \frac{\sigma_{\max}(\mathbf{X})}{\sigma_{\min}(\mathbf{X})},$$

the ratio of the largest and smallest singular values of  $\mathbf{X}$  (Chapter 1). Geometrically,  $\kappa(\mathbf{X})$  is large when some combination of columns of unit coefficient norm is nearly zero. The columns are then close to linearly dependent, and the coordinates of a point in  $\mathbb{C}(\mathbf{X})$  are poorly determined, even though the point itself is not. The singular values of  $\mathbf{X}\mathbf{T}\mathbf{X}$  are the squares of those of  $\mathbf{X}$ , so

$$\kappa(\mathbf{X}\mathbf{T}\mathbf{X}) = \kappa(\mathbf{X})^2.$$

A useful rule of thumb from numerical analysis says that solving a linear system with condition number  $\kappa$  in double precision loses about  $\log_{10} \kappa$  of the roughly 16 available significant digits. The

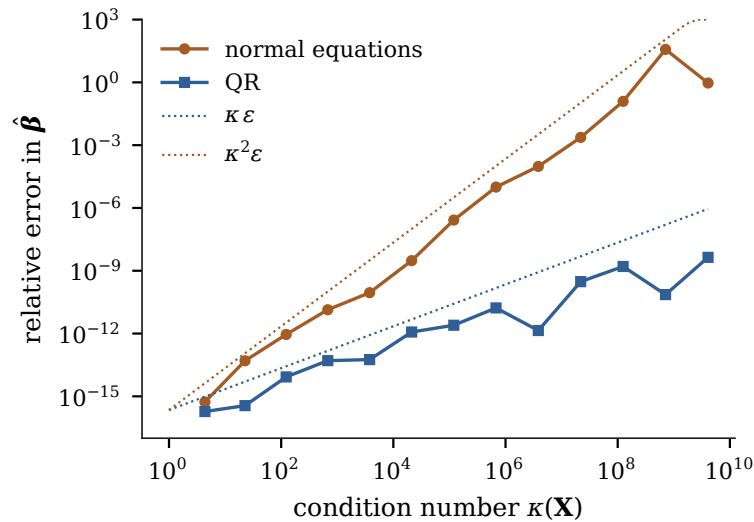


Figure 6.10.1: Accuracy of least squares coefficients for polynomial regressions of degree 1 to 13, against the condition number of the model matrix. The error of the normal equations tracks  $\kappa^2\epsilon$  until it reaches order one. The error of QR tracks  $\kappa\epsilon$ .

normal equations solve a system whose matrix is  $\mathbf{X}^T\mathbf{X}$ , so they lose about  $2\log_{10}\kappa(\mathbf{X})$  digits. QR solves the triangular system  $\mathbb{R}\mathbf{b} = \mathbb{Q}\mathbf{T}\mathbf{y}$ , and  $\kappa(\mathbb{R}) = \kappa(\mathbf{X})$  because  $\mathbb{Q}$  has orthonormal columns, so it loses about  $\log_{10}\kappa(\mathbf{X})$ . The precise perturbation theory has one more subtlety. When the residual is large, the least squares *problem* itself has a sensitivity term proportional to  $\kappa(\mathbf{X})^2\|\hat{\epsilon}\|$ , and no algorithm can avoid it (Higham 2002, ch. 20). QR’s advantage is largest when the model fits well.

#### Example. (Polynomial regression)

The columns  $1, t, t^2, \dots, t^d$  evaluated at 200 equally spaced points in  $[0, 1]$  become nearly dependent as  $d$  grows. To isolate the numerical error, the listing sets  $\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$  exactly for a random  $\boldsymbol{\beta}$ , so that the true least squares solution is  $\boldsymbol{\beta}$  itself. For  $d = 8$  the condition number is  $6.8 \times 10^5$ . The normal equations return  $\boldsymbol{\beta}$  with relative error  $1.0 \times 10^{-5}$ , while QR achieves  $1.7 \times 10^{-11}$ . At  $d = 13$ , with  $\kappa(\mathbf{X}) = 4.1 \times 10^9$ , the normal-equation answer has relative error  $9.3 \times 10^{-1}$  and not one correct digit, while QR still gets  $4.4 \times 10^{-9}$ . Figure 6.10.1 plots the whole sequence against the reference lines  $\kappa\epsilon$  and  $\kappa^2\epsilon$ , where  $\epsilon \approx 2.2 \times 10^{-16}$  is machine precision.

```
import numpy as np
import statsmodels.api as sm

def fit_normal_equations(X, y):
    return np.linalg.solve(X.T @ X, X.T @ y)

def fit_qr(X, y):
    Q, R = np.linalg.qr(X)
    return np.linalg.solve(R, Q.T @ y)  # X = QR, Q has orthonormal columns
                                     # triangular system R b = Q'y
```

[Run this code online](#)

```
rng = np.random.default_rng(10)
n = 200
t = np.linspace(0, 1, n)
```

```

rows = []

for degree in range(1, 14):
    X = np.vander(t, degree + 1, increasing=True)           # 1, t, t^2, ..., t^
    degree
    beta = rng.normal(size=degree + 1)
    y = X @ beta                                           # exact fit: the true
    answer is beta
    kappa = np.linalg.cond(X)
    err_ne = np.linalg.norm(fit_normal_equations(X, y) - beta) / np.linalg.norm(beta)
    err_qr = np.linalg.norm(fit_qr(X, y) - beta) / np.linalg.norm(beta)
    rows.append((degree, kappa, err_ne, err_qr))
    print(f"degree {degree:2d}  cond(X) = {kappa:9.2e}  normal eq. {err_ne:8.1e}
    QR {err_qr:8.1e}")

```

[Run this code online](#)

The polynomial example is extreme by design, but ill-conditioning is common in real data. The macroeconomic series of Longley's classic test problem (in `statsmodels.datasets.longley`) give a model matrix with  $\kappa(\mathbf{X}) \approx 4.86 \times 10^9$ . Then  $\kappa(\mathbf{X}\mathbf{T}\mathbf{X}) \approx 2.36 \times 10^{19}$ , which is larger than  $1/\epsilon$ . The rule of thumb predicts about 6 correct digits from QR and about 0 from the normal equations. Part of that condition number is only a matter of units: one column is a year, another is population in thousands. Rescaling columns (Theorem 6.7.1) changes  $\kappa$  without changing the fit (Exercise (B1)). The part that remains after rescaling reflects genuine near-dependence among the regressors, the subject of Chapter 26.

Two practical lessons follow. First, never compute  $(\mathbf{X}\mathbf{T}\mathbf{X})^{-1}$  just to obtain  $\hat{\beta}$ . Solve with QR, or call a least squares routine that does. Second, reparameterizations that leave  $\mathbb{C}(\mathbf{X})$  unchanged, such as centring, scaling and orthogonal polynomials, are statistically neutral but can be numerically decisive.

### 6.10.3 Rank deficiency and the singular value decomposition

QR with positive diagonal needs full column rank. For rank-deficient or nearly rank-deficient  $\mathbf{X}$ , the most informative tool is the **singular value decomposition**

$$\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}\mathbf{T} = \mathbf{U}_r\mathbf{D}_r\mathbf{V}_r\mathbf{T},$$

where  $\mathbf{U}$  ( $n \times n$ ) and  $\mathbf{V}$  ( $p \times p$ ) are orthogonal,  $\mathbf{D}$  is  $n \times p$  with nonnegative diagonal entries  $\sigma_1 \geq \dots \geq \sigma_r > 0$ , and the subscript  $r$  keeps only the first  $r$  columns. Then:

- $\mathbf{U}_r$  is an orthonormal basis for  $\mathbb{C}(\mathbf{X})$ , so  $\mathbf{M} = \mathbf{U}_r\mathbf{U}_r\mathbf{T}$ ;
- the last  $p - r$  columns of  $\mathbf{V}$  form an orthonormal basis for  $\mathcal{N}(\mathbf{X})$ ;
- the Moore–Penrose inverse is  $\mathbf{X}^+ = \mathbf{V}_r\mathbf{D}_r^{-1}\mathbf{U}_r\mathbf{T}$ , and  $\mathbf{M} = \mathbf{X}\mathbf{X}^+$ .

**Proposition 6.10.1 — Minimum-norm least squares**

$\hat{\beta}^+ = \mathbf{X}^+ \mathbf{y}$  is a least squares estimate, and it has the smallest Euclidean norm among all least squares estimates.

*Proof.*  $\mathbf{X}\mathbf{X}^+ \mathbf{y} = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r \mathbf{T} \mathbf{V}_r \mathbf{D}_r^{-1} \mathbf{U}_r \mathbf{T} \mathbf{y} = \mathbf{U}_r \mathbf{U}_r \mathbf{T} \mathbf{y} = \mathbf{M} \mathbf{y}$ , so  $\hat{\beta}^+$  is a least squares estimate. It lies in  $\mathbb{C}(\mathbf{V}_r) = \mathbb{C}(\mathbf{X}\mathbf{T}) = \mathcal{N}(\mathbf{X})^\perp$ . Every other least squares estimate is  $\hat{\beta}^+ + \mathbf{v}$  with  $\mathbf{v} \in \mathcal{N}(\mathbf{X})$  (Theorem 6.4.2), and  $\|\hat{\beta}^+ + \mathbf{v}\|^2 = \|\hat{\beta}^+\|^2 + \|\mathbf{v}\|^2$ . ■

In Example (Four answers, one fit), the Moore–Penrose row is this minimum-norm solution. In floating point, “ $\sigma_j > 0$ ” becomes “ $\sigma_j$  larger than a tolerance”, and choosing the tolerance decides the numerical rank. That choice is a statistical decision in disguise. Chapter 10 discusses it, together with pivoted QR, which gives most of the SVD’s rank-revealing ability at lower cost.

## 6.10.4 Exercises

### 6.10.4.1 A. Check your understanding

#### Exercise A1.

Prove the four identities in [Eq. 6.10.1](#).

### 6.10.4.2 B. Practice

#### Exercise B1.

Compute  $\kappa(\mathbf{X})$  for the Longley data (with intercept) as given, after scaling every non-intercept column to unit Euclidean norm, and after centring and scaling. Confirm that the fitted values agree to the precision you expect in all three cases. Which part of the original condition number is “just units”?

#### Exercise B2.

Implement classical Gram–Schmidt as in Proposition 6.2.2, and the *modified* version that subtracts each projection from the working vector before computing the next inner product. For the degree-10 polynomial design of [Example \(Polynomial regression\)](#), compare  $\|\mathbf{Q}\mathbf{T}\mathbf{Q} - \mathbf{I}\|$  for the two and for `numpy.linalg.qr`.

### 6.10.4.3 C. Going deeper

#### Exercise C1.

Let  $\mathbf{X} = [\mathbf{1}, \mathbf{x}]$  with  $\bar{x} = c$  and  $S_{xx} = s^2 n$ . Find  $\kappa(\mathbf{X})$  as a function of  $c$ ,  $s$  and  $n$ , and show that it grows without bound as  $c \rightarrow \infty$  with  $s$  fixed. Show that centring the regressor, a reparameterization, reduces the condition number to  $\max(1, s)/\min(1, s)$ .

*Solution.* Write  $\mathbf{x} = c\mathbf{1} + \mathbf{d}$  with  $\mathbf{d} \perp \mathbf{1}$  and  $\|\mathbf{d}\|^2 = ns^2$ . Then  $\mathbf{X}\mathbf{T}\mathbf{X} = n \begin{pmatrix} 1 & c \\ c & c^2 + s^2 \end{pmatrix}$ . The matrix in parentheses has trace  $T = 1 + c^2 + s^2$  and determinant  $s^2$ , so its eigenvalues are  $\lambda_{\pm} = (T \pm \sqrt{T^2 - 4s^2})/2$ , with  $\lambda_+\lambda_- = s^2$  and  $\lambda_+ \geq T/2$ . Hence

$$\kappa(\mathbf{X})^2 = \frac{\lambda_+}{\lambda_-} = \frac{\lambda_+^2}{s^2} \geq \frac{T^2}{4s^2} \rightarrow \infty \quad \text{as } c \rightarrow \infty.$$

After centring,  $\mathbf{X} = [\mathbf{1}, \mathbf{d}]$  has orthogonal columns of lengths  $\sqrt{n}$  and  $s\sqrt{n}$ , so its singular values are these lengths and  $\kappa = \max(1, s)/\min(1, s)$ .